

SMART DELIVERY OPERATIONS ANALYTICS SYSTEM

SQL PROJECT REPORT

Submitted by
Pothuru Kavya Sree

Technology:

- MySQL 8.0
- MySQL Workbench
- SQL

Table of Contents:

1. Introduction
2. Project Objectives
3. Dataset Description
4. Database Design (ER Diagram)
5. Database Schema
6. Data Import and Validation
7. Data Cleaning
8. SQL Analysis
9. Business Insights
10. Recommendations
11. Conclusion

1. Introduction

The Smart Delivery Operations Analytics System is a SQL-based analytical project designed to manage and analyze food delivery operations. The project demonstrates database design, data cleaning, SQL querying, and business reporting using a real-world delivery dataset.

The system stores information about customers, restaurants, delivery partners, orders, payments, ratings, and promotions. SQL is used to generate insights that help management improve operational efficiency, customer satisfaction, and business profitability.

2. Project Objectives

The objectives of this project are:

- Design a normalized relational database.
- Import CSV files into MySQL.
- Validate imported data.
- Clean inconsistent and missing data.
- Perform business analysis using SQL.
- Generate business insights.
- Recommend solutions to improve operations.

3. Dataset Description

The project consists of seven interconnected tables used to analyze delivery operations, customer behaviour, revenue, and partner performance.

Table	Description
Customers	Stores customer information including name, age, gender, city, and signup date.
Restaurants	Stores restaurant details such as restaurant name, cuisine type, and city.
Delivery Partners	Contains delivery partner information.
Orders	Stores order details including order time, delivery time, order amount, and status.
Payments	Contains payment transaction details for each order.
Ratings	Stores customer ratings and reviews for completed orders.
Customer Behaviour	Derived analytics table containing customer behaviour metrics such as total orders, total spending, average order value, days since last order, and churn status.

Total records imported:

Customers – 1,000
Restaurants – 100
Delivery Partners – 200
Orders – 10,000
Payments – 10,000
Ratings – 7,000
Customer Behaviour – 1000

Overall dataset size: 29,300+ records

4. Database Design (ER Diagram)

The database was designed using normalization principles.

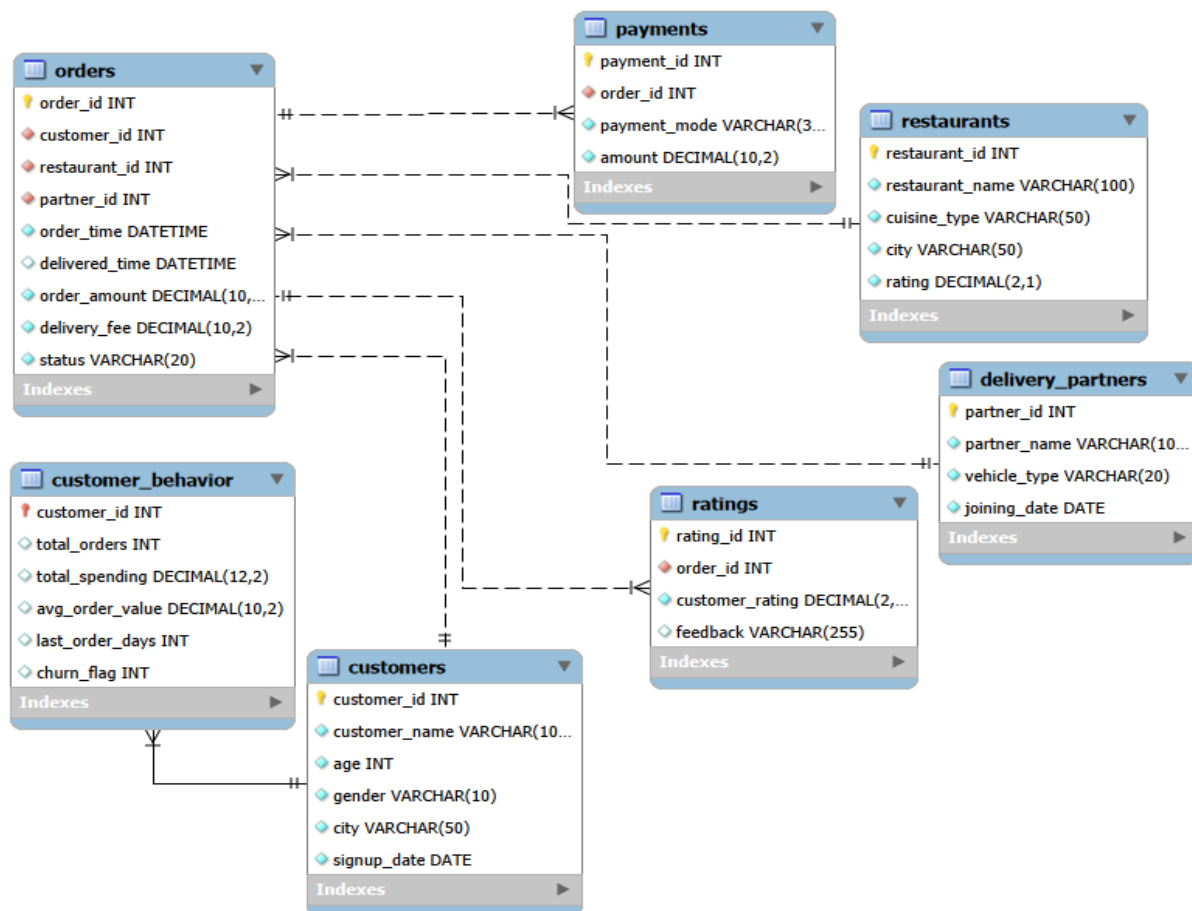
Entities

Customers
Restaurants
Delivery Partners
Orders

Payments
Ratings
Customer Behaviour

Relationships

One Customer → Many Orders
One Restaurant → Many Orders
One Delivery Partner → Many Orders
One Order → One Payment
One Order → One Rating



5. Database Schema

Each table was created using SQL DDL.
The schema includes:

Primary Keys
Foreign Keys
NOT NULL Constraints
CHECK Constraints
Appropriate Data Types

Create Database:

```
create database smart_delivery;  
use smart_delivery;
```

Create Table:

1. CUSTOMERS

```
CREATE TABLE customers (  
    customer_id INT NOT NULL,  
    customer_name VARCHAR(100) NOT NULL,  
    age INT NOT NULL,  
    gender VARCHAR(10) NOT NULL,  
    city VARCHAR(50) NOT NULL,  
    signup_date DATE NOT NULL,  
  
    CONSTRAINT pk_customers  
        PRIMARY KEY (customer_id),  
  
    CONSTRAINT chk_customer_age  
        CHECK (age BETWEEN 18 AND 100),  
  
    CONSTRAINT chk_customer_gender  
        CHECK (gender IN ('Male','Female','Other'))  
);
```

2. RESTAURANTS

```
CREATE TABLE restaurants (  
    restaurant_id INT NOT NULL,  
    restaurant_name VARCHAR(100) NOT NULL,  
    cuisine_type VARCHAR(50) NOT NULL,  
    city VARCHAR(50) NOT NULL,  
    rating DECIMAL(2,1) NOT NULL,  
  
    CONSTRAINT pk_restaurants  
        PRIMARY KEY (restaurant_id),  
  
    CONSTRAINT chk_restaurant_rating  
        CHECK (rating BETWEEN 1.0 AND 5.0)  
);
```

3. DELIVERY_PARTNERS

```
CREATE TABLE delivery_partners (  
    partner_id INT NOT NULL,  
    partner_name VARCHAR(100) NOT NULL,  
    vehicle_type VARCHAR(20) NOT NULL,  
    joining_date DATE NOT NULL,
```

```
CONSTRAINT pk_delivery_partners
    PRIMARY KEY (partner_id),

CONSTRAINT chk_vehicle_type
    CHECK (vehicle_type IN
        ('Bike','Scooter','Bicycle','Electric Bike'))
);
```

4. ORDERS

```
CREATE TABLE orders (
    order_id INT NOT NULL,
    customer_id INT NOT NULL,
    restaurant_id INT NOT NULL,
    partner_id INT NOT NULL,
    order_time DATETIME NOT NULL,
    delivered_time DATETIME,
    order_amount DECIMAL(10,2) NOT NULL,
    delivery_fee DECIMAL(10,2) NOT NULL,
    status VARCHAR(20) NOT NULL,

    CONSTRAINT pk_orders
        PRIMARY KEY (order_id),

    CONSTRAINT fk_orders_customer
        FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id),

    CONSTRAINT fk_orders_restaurant
        FOREIGN KEY (restaurant_id)
        REFERENCES restaurants(restaurant_id),

    CONSTRAINT fk_orders_partner
        FOREIGN KEY (partner_id)
        REFERENCES delivery_partners(partner_id),

    CONSTRAINT chk_order_amount
        CHECK (order_amount > 0),

    CONSTRAINT chk_delivery_fee
        CHECK (delivery_fee >= 0),

    CONSTRAINT chk_order_status
        CHECK (status IN
            ('Delivered','Cancelled','Pending'))
);
```

5. PAYMENTS

```
CREATE TABLE payments (
    payment_id INT NOT NULL,
```

```

order_id INT NOT NULL,
payment_mode VARCHAR(30) NOT NULL,
amount DECIMAL(10,2) NOT NULL,

CONSTRAINT pk_payments
    PRIMARY KEY (payment_id),

CONSTRAINT fk_payments_order
    FOREIGN KEY (order_id)
    REFERENCES orders(order_id),

CONSTRAINT chk_payment_amount
    CHECK (amount > 0),

CONSTRAINT chk_payment_mode
    CHECK (payment_mode IN
('UPI',
 'Credit Card',
 'Debit Card',
 'Cash on Delivery',
 'Wallet'))
);

```

6. RATINGS

```

CREATE TABLE ratings (
    rating_id INT NOT NULL,
    order_id INT NOT NULL,
    customer_rating DECIMAL(2,1) NOT NULL,
    feedback VARCHAR(255),

    CONSTRAINT pk_ratings
        PRIMARY KEY (rating_id),

    CONSTRAINT fk_ratings_order
        FOREIGN KEY (order_id)
        REFERENCES orders(order_id),

    CONSTRAINT chk_customer_rating
        CHECK (customer_rating BETWEEN 1.0 AND 5.0)
);

```

7. CUSTOMER_BEHAVIOR

```

CREATE TABLE customer_behavior (
    customer_id INT NOT NULL,
    total_orders INT DEFAULT 0,
    total_spending DECIMAL(12,2) DEFAULT 0,
    avg_order_value DECIMAL(10,2),
    last_order_days INT,
    churn_flag INT,

```

```

CONSTRAINT pk_customer_behavior
    PRIMARY KEY (customer_id),

CONSTRAINT fk_cb_customer
    FOREIGN KEY (customer_id)
    REFERENCES customers(customer_id),

CONSTRAINT chk_total_orders
    CHECK (total_orders >= 0),

CONSTRAINT chk_total_spending
    CHECK (total_spending >= 0),

CONSTRAINT chk_avg_order_value
    CHECK (avg_order_value >= 0),

CONSTRAINT chk_last_order_days
    CHECK (last_order_days >= 0),

CONSTRAINT chk_churn_flag
    CHECK (churn_flag IN (0,1))
);

```

6. Data Import and Validation

All CSV files were successfully imported into MySQL using the Import Wizard.

```

LOAD DATA LOCAL INFILE
'C:/Users/LENOVO/OneDrive/Desktop/D13/sql/D13_SQL_Project/customers.csv'
INTO TABLE customers
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS;

```

```

LOAD DATA LOCAL INFILE
'C:/Users/LENOVO/OneDrive/Desktop/D13/sql/D13_SQL_Project/restaurants.csv'
INTO TABLE restaurants
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS;

```

```

LOAD DATA LOCAL INFILE
'C:/Users/LENOVO/OneDrive/Desktop/D13/sql/D13_SQL_Project/delivery_partners.csv'
INTO TABLE delivery_partners
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'

```

```
IGNORE 1 ROWS;
```

```
LOAD DATA LOCAL INFILE
```

```
'C:/Users/LENOVO/OneDrive/Desktop/D13/sql/D13_SQL_Project/orders.csv'
```

```
INTO TABLE orders
```

```
FIELDS TERMINATED BY ','
```

```
ENCLOSED BY ''
```

```
LINES TERMINATED BY '\r\n'
```

```
IGNORE 1 ROWS;
```

```
LOAD DATA LOCAL INFILE
```

```
'C:/Users/LENOVO/OneDrive/Desktop/D13/sql/D13_SQL_Project/payments.csv'
```

```
INTO TABLE payments
```

```
FIELDS TERMINATED BY ','
```

```
ENCLOSED BY ''
```

```
LINES TERMINATED BY '\r\n'
```

```
IGNORE 1 ROWS;
```

```
LOAD DATA LOCAL INFILE
```

```
'C:/Users/LENOVO/OneDrive/Desktop/D13/sql/D13_SQL_Project/ratings.csv'
```

```
INTO TABLE ratings
```

```
FIELDS TERMINATED BY ','
```

```
ENCLOSED BY ''
```

```
LINES TERMINATED BY '\r\n'
```

```
IGNORE 1 ROWS;
```

```
LOAD DATA LOCAL INFILE
```

```
'C:/Users/LENOVO/OneDrive/Desktop/D13/sql/D13_SQL_Project/customer_behavior.csv'
```

```
INTO TABLE customer_behavior
```

```
FIELDS TERMINATED BY ','
```

```
ENCLOSED BY ''
```

```
LINES TERMINATED BY '\r\n'
```

```
IGNORE 1 ROWS;
```

Validate Record Counts:

Validation was performed after each import using:

```
SELECT COUNT(*) FROM customers;
```

```
SELECT COUNT(*) FROM restaurants;
```

```
SELECT COUNT(*) FROM delivery_partners;
```

```
SELECT COUNT(*) FROM orders;
```

```
SELECT COUNT(*) FROM payments;
```

```
SELECT COUNT(*) FROM ratings;
```

```
SELECT COUNT(*) FROM customer_behavior;
```

The record counts matched the source CSV files, confirming successful import.

7. Data Cleaning

Several data quality issues were identified and resolved.

NULL Values

```
SELECT * FROM customers
WHERE customer_name IS NULL
    OR city IS NULL
    OR signup_date IS NULL;
```

```
SELECT * FROM restaurants
WHERE restaurant_name IS NULL
    OR city IS NULL
    OR cuisine_type IS NULL;
```

```
SELECT * FROM delivery_partners
WHERE partner_name IS NULL
    OR vehicle_type IS NULL;
```

```
SELECT *FROM payments
WHERE payment_mode IS NULL
    OR amount IS NULL;
```

```
SELECT *FROM ratings
WHERE customer_rating IS NULL;
```

```
SELECT *FROM customer_behavior
WHERE total_orders IS NULL
    OR total_spending IS NULL;
```

```
SELECT *FROM orders
WHERE delivered_time IS NULL;
```

```
UPDATE orders
SET delivered_time = NULL
WHERE status IN ('Pending', 'Cancelled');
```

```
SELECT status, COUNT(*) AS total_orders
FROM orders
WHERE delivered_time IS NULL
GROUP BY status;
```

```
SELECT *FROM orders
WHERE status='Delivered'
AND delivered_time IS NULL;
```

Missing delivery timestamps were found only for Pending and Cancelled orders.

Duplicate Records

Duplicates were identified using:

```
SELECT customer_id, COUNT(*)
FROM customers
GROUP BY customer_id
```

```
HAVING COUNT(*)>1;
```

```
SELECT restaurant_id, COUNT(*)  
FROM restaurants  
GROUP BY restaurant_id  
HAVING COUNT(*)>1;
```

```
SELECT order_id, COUNT(*)  
FROM orders  
GROUP BY order_id  
HAVING COUNT(*)>1;
```

```
SELECT payment_id, COUNT(*)  
FROM payments  
GROUP BY payment_id  
HAVING COUNT(*)>1;
```

```
SELECT partner_id, COUNT(*)  
FROM delivery_partners  
GROUP BY partner_id  
HAVING COUNT(*)>1;
```

No duplicate primary keys were found.

Rating Validation

Ratings were validated using:

```
SELECT *  
FROM restaurants  
WHERE rating NOT BETWEEN 1 AND 5;
```

```
SELECT *  
FROM ratings  
WHERE customer_rating NOT BETWEEN 1 AND 5;
```

All ratings were within the valid range.

City Standardization

City names were validated using:

```
SELECT DISTINCT city  
FROM customers  
ORDER BY city;
```

```
SELECT DISTINCT city  
FROM restaurants  
ORDER BY city;
```

Validate Referential Integrity

Orders → Customers

```
SELECT *  
FROM orders o  
LEFT JOIN customers c  
ON o.customer_id=c.customer_id  
WHERE c.customer_id IS NULL;
```

Orders → Restaurants

```
SELECT *  
FROM orders o  
LEFT JOIN restaurants r  
ON o.restaurant_id=r.restaurant_id  
WHERE r.restaurant_id IS NULL;
```

Orders → Delivery Partners

```
SELECT *  
FROM orders o  
LEFT JOIN delivery_partners d  
ON o.partner_id=d.partner_id  
WHERE d.partner_id IS NULL;
```

Payments → Orders

```
SELECT *  
FROM payments p  
LEFT JOIN orders o  
ON p.order_id=o.order_id  
WHERE o.order_id IS NULL;
```

Ratings → Orders

```
SELECT *  
FROM ratings r  
LEFT JOIN orders o  
ON r.order_id=o.order_id  
WHERE o.order_id IS NULL;
```

Data Validation

Final validation confirmed:

- No duplicate primary keys.
- Valid foreign key relationships.
- Valid ratings.
- Correct delivery timestamps.
- Consistent city names.

8. SQL Analysis

More than 40 SQL queries were executed.

The queries covered:

Delivery Performance

Examples:

Longest average delivery time

Fastest delivery partners

Cities with delivery delays

Cancellation Analysis

Examples:

Highest cancellation restaurants

Peak cancellation day

Peak cancellation time

Revenue Analysis

Examples:

Monthly revenue

Top restaurants

Top cities

Payment methods

Customer Analysis

Examples:

Top spending customers

Inactive customers

Frequent customers

Churn analysis

Delivery Partner Analysis

Examples:

Fastest partners

Highest completed deliveries

Average ratings

Partner performance

The project includes SQL concepts such as:

SELECT

GROUP BY

HAVING

JOIN

Subqueries

CTE

Window Functions

Stored Procedures

9. Business Insights

Delivery Delays

Q1 Which restaurants are associated with the longest average delivery times?

SELECT

r.restaurant_id,

```

r.restaurant_name,

ROUND(AVG(TIMESTAMPDIFF(MINUTE,

    o.order_time,

    o.delivered_time)), 2) AS avg_delivery_time_minutes

FROM orders o

INNER JOIN restaurants r

    ON o.restaurant_id = r.restaurant_id

WHERE o.status = 'Delivered'

GROUP BY r.restaurant_id, r.restaurant_name

ORDER BY avg_delivery_time_minutes DESC;

```

restaurant_id	restaurant_name	avg_delivery_time_minutes
86	Bombay Hub	61.86
13	Royal Delight	60.67
29	Royal Express	59.43
61	Star Express	59.41
2	Star Junction	58.65
87	Royal Point	58.58
79	Fresh Delight	58.56
92	Royal Hub	57.77
22	Golden Corner	57.77
96	Bombay Point	57.57

Business Insight

Restaurants with the highest average delivery times may have slower food preparation or inefficient order handling. These delays negatively affect customer satisfaction.

Recommendation

- Improve kitchen preparation efficiency.
- Monitor restaurant performance regularly.
- Optimize order processing.

Q2 Which delivery partners have the highest average time per order?

```

SELECT

d.partner_id,

d.partner_name,

ROUND(AVG(TIMESTAMPDIFF(MINUTE,

    o.order_time,

    o.delivered_time)),2) AS average_delivery_time_minutes

```

```

FROM orders o

INNER JOIN delivery_partners d

    ON o.partner_id = d.partner_id

WHERE o.status = 'Delivered'

GROUP BY

    d.partner_id,

    d.partner_name

ORDER BY average_delivery_time_minutes DESC;

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
partner_id	partner_name	average_delivery_time_minutes	
178	Harish Kumar	62.21	
49	Arumugam Nair	62.20	
189	Thiru Pillai	62.10	
167	Harish Pillai	61.64	
42	Gopal Sharma	61.56	
177	Suresh Kumar	61.23	
101	Selvam Nair	61.14	
41	Manoj Rao	60.69	
53	Vinod Singh	60.66	
197	Selvam Reddy	60.56	

Business Insight

Some delivery partners consistently take longer to complete deliveries, indicating possible route or workload inefficiencies.

Recommendation

- Provide route optimization.
- Balance delivery assignments.
- Offer additional training.

Q3 Which cities experience the most delivery delays?

```

SELECT

    c.city,

    ROUND(AVG(TIMESTAMPDIFF(MINUTE,

        o.order_time,

        o.delivered_time)),2) AS average_delivery_delay_minutes

FROM orders o

INNER JOIN customers c

```

```

ON o.customer_id = c.customer_id

WHERE o.status = 'Delivered'

GROUP BY c.city

ORDER BY average_delivery_delay_minutes DESC;

```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	city	average_delivery_delay_minutes			
▶	Chennai	55.43			
	Surat	55.38			
	Kolkata	55.32			
	Ahmedabad	55.11			
	Hyderabad	54.91			
	Pune	54.87			
	Delhi	54.81			
	Mumbai	54.58			
	Jaipur	54.18			
	Bangalore	53.40			

Business Insight

Cities with higher average delivery times may experience traffic congestion or insufficient delivery partner availability.

Recommendation

- Increase delivery partners in those cities.
- Improve delivery planning.
- Analyze peak-hour demand.

Q4 Are delivery delays increasing or decreasing over time?

```

SELECT

YEAR(o.order_time) AS year,

MONTH(o.order_time) AS month,

COUNT(o.order_id) AS total_orders,

ROUND(AVG(TIMESTAMPDIFF(MINUTE,

    o.order_time,

    o.delivered_time)),2) AS average_delivery_time_minutes

FROM orders o

WHERE o.status = 'Delivered'

GROUP BY

YEAR(o.order_time),

```

MONTH(o.order_time)

ORDER BY

year,month;

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
year	month	total_orders	average_delivery_time_minutes
2023	1	359	54.74
2023	2	316	57.52
2023	3	365	54.37
2023	4	368	53.46
2023	5	350	55.03
2023	6	353	53.79
2023	7	357	54.37
2023	8	333	55.41
2023	9	335	54.13
2023	10	345	54.26

Business Insight

Monthly delivery trends reveal whether operational performance is improving or declining over time.

Recommendation

- Continuously monitor delivery KPIs.
- Investigate months with increasing delays.
- Review operational processes regularly.

Order Cancellations

Q1 Which restaurants receive the highest volume of cancellations?

SELECT

r.restaurant_id,

r.restaurant_name,

COUNT(o.order_id) AS total_cancellations

FROM orders o

INNER JOIN restaurants r

ON o.restaurant_id = r.restaurant_id

WHERE o.status = 'Cancelled'

GROUP BY

r.restaurant_id,

r.restaurant_name

ORDER BY total_cancellations DESC;

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
restaurant_id	restaurant_name	total_cancellations	
98	Classic Point	22	
88	Royal Corner	20	
49	Urban Kitchen	19	
99	Royal Delight	19	
47	Classic Point	19	
44	Fresh Bites	19	
60	Star Delight	19	
67	Royal Kitchen	18	
7	Taste Junction	18	
58	Star Delight	17	

Business Insight

Restaurants with frequent cancellations may face inventory shortages, preparation delays, or operational issues.

Recommendation

- Improve inventory management.
- Monitor restaurant performance.
- Reduce food preparation time.

Q2 Which cities have the highest cancellation rates by percentage?

SELECT

c.city,

COUNT(*) AS total_orders,

SUM(o.status = 'Cancelled') AS cancelled_orders,

ROUND((SUM(o.status = 'Cancelled') * 100.0) / COUNT(*),2) AS cancellation_rate_percentage

FROM orders o

INNER JOIN customers c

ON o.customer_id = c.customer_id

GROUP BY c.city

ORDER BY cancellation_rate_percentage DESC;

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	city	total_orders	cancelled_orders	cancellation_rate_percentage
▶	Delhi	1106	147	13.29
	Kolkata	928	122	13.15
	Mumbai	831	108	13.00
	Jaipur	952	120	12.61
	Ahmedabad	1138	139	12.21
	Bangalore	1070	126	11.78
	Surat	1007	113	11.22
	Pune	1154	128	11.09
	Hyderabad	892	98	10.99
	Chennai	922	94	10.20

Business Insight

Higher cancellation rates in certain cities indicate location-specific operational challenges.

Recommendation

- Increase delivery capacity.
- Improve customer communication.
- Analyze local operational issues.

Q3 Is there a correlation between delivery delays and cancellations?

SELECT

r.restaurant_name,

ROUND(AVG(TIMESTAMPDIFF(MINUTE,

o.order_time,

o.delivered_time)),2) AS average_delivery_time_minutes,

SUM(o.status = 'Cancelled') AS cancelled_orders

FROM orders o

INNER JOIN restaurants r

ON o.restaurant_id = r.restaurant_id

GROUP BY

r.restaurant_id,

r.restaurant_name

ORDER BY

cancelled_orders DESC,

average_delivery_time_minutes DESC;

Result Grid			
Filter Rows:		Export:	Wrap Cell Content:
restaurant_name	average_delivery_time_minutes	cancelled_orders	
Classic Point	54.67	22	
Royal Corner	54.16	20	
Classic Point	57.41	19	
Fresh Bites	54.66	19	
Star Delight	54.10	19	
Royal Delight	53.79	19	
Urban Kitchen	53.63	19	
Taste Junction	56.13	18	
Royal Kitchen	54.92	18	
Star Junction	58.65	17	

Business Insight

Restaurants with longer delivery times generally experience more cancellations, indicating that delays contribute to order cancellations.

Recommendation

- Reduce delivery times.
- Improve dispatch efficiency.
- Monitor high-risk restaurants.

Q4 On which days and at which times do cancellations peak?

SELECT

DAYNAME(o.order_time) AS day_of_week,

HOUR(o.order_time) AS order_hour,

COUNT(*) AS total_cancellations

FROM orders o

WHERE o.status = 'Cancelled'

GROUP BY

DAYNAME(o.order_time),

HOUR(o.order_time)

ORDER BY total_cancellations DESC;

Result Grid				Filter Rows:	Export:	Wrap Cell Content:
	day_of_week	order_hour	total_cancellations			
▶	Monday	16	16			
	Saturday	17	13			
	Monday	22	13			
	Friday	4	12			
	Monday	0	12			
	Saturday	11	11			
	Wednesday	18	11			
	Friday	16	11			
	Saturday	16	11			
	Tuesday	20	11			

Business Insight

Cancellations increase during specific days and peak hours due to higher order volumes.

Recommendation

- Increase staffing during peak hours.
- Improve resource allocation.
- Prepare restaurants for high-demand periods.

Revenue Optimisation

Q1 Which restaurants generate the highest total revenue?

SELECT

r.restaurant_id,

r.restaurant_name,

ROUND(SUM(p.amount), 2) AS total_revenue

FROM restaurants r

INNER JOIN orders o

ON r.restaurant_id = o.restaurant_id

INNER JOIN payments p

ON o.order_id = p.order_id

WHERE o.status = 'Delivered'

GROUP BY

r.restaurant_id,

r.restaurant_name

ORDER BY total_revenue DESC;

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
restaurant_id	restaurant_name	total_revenue	
82	Taste Kitchen	83687.47	
2	Star Junction	83628.04	
75	Golden Express	83520.17	
34	Spice Junction	83159.87	
85	Urban Express	81968.78	
69	Desi Corner	79620.25	
56	Star Express	78492.41	
92	Royal Hub	78376.77	
27	Desi Hub	77961.49	
84	Bombay Point	77954.17	

Business Insight

A small number of restaurants contribute significantly to total company revenue.

Recommendation

- Strengthen partnerships with top restaurants.
- Promote lower-performing restaurants.

Q2 Which cities contribute the most to overall company revenue?

SELECT

c.city,

ROUND(SUM(p.amount), 2) AS total_revenue

FROM customers c

INNER JOIN orders o

ON c.customer_id = o.customer_id

INNER JOIN payments p

ON o.order_id = p.order_id

WHERE o.status = 'Delivered'

GROUP BY c.city

ORDER BY total_revenue DESC;

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	city	total_revenue			
▶	Pune	773932.42			
	Ahmedabad	745625.76			
	Delhi	739726.32			
	Bangalore	716324.94			
	Surat	687716.23			
	Jaipur	654478.72			
	Chennai	628830.50			
	Hyderabad	614614.19			
	Kolkata	604018.22			
	Mumbai	539489.89			

Business Insight

Revenue is concentrated in a few cities, indicating strong market demand in those locations.

Recommendation

- Expand services in high-revenue cities.
- Increase marketing efforts.

Q3 Which payment methods are most popular and most valuable?

SELECT

p.payment_mode,

COUNT(p.payment_id) AS total_transactions,

ROUND(SUM(p.amount), 2) AS total_revenue

FROM payments p

INNER JOIN orders o

ON p.order_id = o.order_id

WHERE o.status = 'Delivered'

GROUP BY p.payment_mode

ORDER BY total_revenue DESC;

Result Grid				Filter Rows:	Export:	Wrap Cell Content:
	payment_mode	total_transactions	total_revenue			
▶	UPI	1740	1430438.59			
	Credit Card	1647	1353885.85			
	Wallet	1644	1345626.43			
	Debit Card	1615	1320232.49			
	Cash on Delivery	1551	1254573.83			

Business Insight

Digital payment methods generate the highest transaction volume and revenue.

Recommendation

- Promote digital payments.
- Offer payment-based cashback.

Q4 What are the monthly revenue trends over the past year?

WITH MonthlyRevenue AS

```
(
SELECT
    DATE_FORMAT(o.order_time, '%M %Y') AS month,
    MIN(o.order_time) AS first_order_date,
    ROUND(SUM(p.amount), 2) AS monthly_revenue
FROM orders o
INNER JOIN payments p
    ON o.order_id = p.order_id
WHERE o.status = 'Delivered'
GROUP BY DATE_FORMAT(o.order_time, '%M %Y')
)
SELECT
    month,
    monthly_revenue
FROM MonthlyRevenue
ORDER BY first_order_date;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	month	monthly_revenue			
▶	January 2023	297729.21			
	February 2023	247299.62			
	March 2023	303344.33			
	April 2023	309232.99			
	May 2023	277518.18			
	June 2023	277367.28			
	July 2023	293469.18			
	August 2023	276507.46			
	September 2023	280572.69			
	October 2023	277370.88			

Business Insight

Monthly revenue trends identify seasonal fluctuations and business growth patterns.

Recommendation

- Launch promotions during low-revenue months.
- Prepare additional resources during peak periods.

Customer Retention

Q1 Who are the top 10 most valuable customers by lifetime spending?

WITH CustomerSpending AS

```
(
SELECT
    c.customer_id,
    c.customer_name,
    ROUND(SUM(p.amount), 2) AS lifetime_spending
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
INNER JOIN payments p
    ON o.order_id = p.order_id
WHERE o.status = 'Delivered'
GROUP BY
    c.customer_id,
    c.customer_name
```

)

SELECT

customer_id,
customer_name,
lifetime_spending

FROM CustomerSpending

ORDER BY lifetime_spending DESC

LIMIT 10;

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
customer_id	customer_name	lifetime_spending	
691	Kiran Shah	16833.84	
512	Amit Verma	15777.08	
759	Naveen Pillai	15394.94	
978	Naveen Gupta	14999.88	
815	Vijay Bose	14937.51	
59	Vikram Rao	14596.78	
191	Manish Chopra	14338.65	
387	Divya Verma	13618.53	
764	Riya Mehta	13324.93	
360	Swati Mehta	13232.11	

Business Insight

A small group of customers contributes a significant share of overall revenue.

Recommendation

- Introduce loyalty rewards.
- Offer exclusive benefits.

Q2 Which customers order most frequently, and what is their average order value?

SELECT

c.customer_id,
c.customer_name,
COUNT(o.order_id) AS total_orders,
ROUND(AVG(o.order_amount), 2) AS average_order_value

FROM customers c

INNER JOIN orders o

ON c.customer_id = o.customer_id

WHERE o.status = 'Delivered'

GROUP BY

c.customer_id,

c.customer_name

ORDER BY total_orders DESC;



The screenshot shows a 'Result Grid' window with a toolbar containing 'Filter Rows', 'Export', and 'Wrap Cell Content' options. The grid displays a table with 5 columns: customer_id, customer_name, total_orders, and average_order_value. The data is sorted by total_orders in descending order. The first row is highlighted with a mouse cursor.

	customer_id	customer_name	total_orders	average_order_value
▶	387	Divya Verma	20	646.15
	512	Amit Verma	18	834.75
	691	Kiran Shah	17	952.48
	815	Vijay Bose	17	846.11
	191	Manish Chopra	17	812.72
	759	Naveen Pillai	17	874.22
	330	Swati Das	16	754.16
	252	Pallavi Kumar	16	660.17
	623	Nisha Verma	16	767.30
	811	Sneha Agarwal	15	834.94

Business Insight

Frequent customers demonstrate strong engagement and higher lifetime value.

Recommendation

- Reward repeat customers.
- Encourage larger order values through bundle offers.

Q3 Which customers have gone inactive in the last 90 days?

SELECT

c.customer_id,

c.customer_name,

MAX(o.order_time) AS last_order_date

FROM customers c

INNER JOIN orders o

ON c.customer_id = o.customer_id

WHERE o.status = 'Delivered'

GROUP BY

c.customer_id,

c.customer_name

HAVING MAX(o.order_time) < DATE_SUB(CURDATE(), INTERVAL 90 DAY)

ORDER BY last_order_date;

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
customer_id	customer_name	last_order_date	
477	Nisha Das	2023-05-21 00:32:20	
847	Rajesh Pillai	2023-08-10 13:11:59	
16	Rohan Das	2023-08-17 11:44:54	
706	Rohan Kumar	2023-09-16 23:50:41	
514	Kiran Singh	2023-10-18 18:37:27	
76	Rekha Gupta	2023-10-21 16:40:10	
968	Naveen Shah	2023-10-23 01:08:18	
44	Asha Patel	2023-10-25 09:08:45	
735	Arun Chopra	2023-11-08 16:28:16	
835	Naveen Shah	2023-11-09 02:42:09	

Business Insight

Inactive customers are at risk of churning and require re-engagement.

Recommendation

- Send personalized promotions.
- Offer special discounts.

Q4 What behavioural patterns signal that a customer is about to churn?

SELECT

c.customer_id,

c.customer_name,

COUNT(o.order_id) AS total_orders,

ROUND(AVG(o.order_amount), 2) AS average_order_value,

MAX(o.order_time) AS last_order_date

FROM customers c

INNER JOIN orders o

ON c.customer_id = o.customer_id

GROUP BY

c.customer_id,

c.customer_name

ORDER BY last_order_date ASC;

Result Grid

 Filter Rows:

 Export:

 Wrap Cell Content:

	customer_id	customer_name	total_orders	average_order_value	last_order_date
▶	706	Rohan Kumar	4	832.58	2023-09-20 13:43:09
	968	Naveen Shah	9	616.18	2023-10-23 01:08:18
	300	Pooja Pillai	7	745.64	2023-11-19 18:53:29
	477	Nisha Das	6	1057.67	2023-11-22 08:35:35
	16	Rohan Das	7	972.71	2024-01-01 04:28:49
	979	Sneha Reddy	9	682.60	2024-01-21 15:37:02
	766	Arjun Gupta	5	889.68	2024-01-23 20:44:44
	187	Naveen Chopra	4	865.37	2024-02-03 10:38:15
	671	Rekha Nair	8	697.05	2024-02-05 12:13:36
	699	Meera Rao	5	667.19	2024-02-13 17:32:09

Business Insight

Customers with fewer orders, lower spending, and long periods of inactivity are more likely to churn.

Recommendation

- Identify at-risk customers early.
- Launch targeted retention campaigns.

Delivery Partner Performance

Q1 Who are the fastest delivery partners based on average delivery time?

SELECT

d.partner_id,

d.partner_name,

ROUND(AVG(TIMESTAMPDIFF(MINUTE,

o.order_time,

o.delivered_time)),2) AS average_delivery_time_minutes

FROM orders o

INNER JOIN delivery_partners d

ON o.partner_id = d.partner_id

WHERE o.status = 'Delivered'

GROUP BY

d.partner_id,

d.partner_name

ORDER BY average_delivery_time_minutes ASC;

Result Grid				Filter Rows:	Export:	Wrap Cell Content:
	partner_id	partner_name	average_delivery_time_minutes			
▶	117	Surya Yadav	46.60			
	74	Gopal Gowda	47.41			
	16	Selvam Gowda	47.93			
	38	Deepak Gowda	48.69			
	142	Manoj Patel	48.72			
	120	Arumugam Pillai	48.90			
	149	Deepak Reddy	49.76			
	34	Mani Gowda	49.85			
	170	Deepak Gowda	49.92			
	111	Prakash Yadav	50.00			

Business Insight

Fast delivery partners consistently complete orders quickly, improving customer satisfaction.

Recommendation

- Reward top-performing partners.
- Share best practices with the team.

Q2 Which partners handle the highest volume of successful deliveries?

SELECT

d.partner_id,

d.partner_name,

COUNT(o.order_id) AS successful_deliveries

FROM orders o

INNER JOIN delivery_partners d

ON o.partner_id = d.partner_id

WHERE o.status = 'Delivered'

GROUP BY

d.partner_id,

d.partner_name

ORDER BY successful_deliveries DESC;

Result Grid				Filter Rows:	Export:	Wrap Cell Content:
	partner_id	partner_name	successful_deliveries			
▶	165	Surya Gowda	57			
	174	Harish Pillai	55			
	197	Selvam Reddy	55			
	198	Suresh Yadav	54			
	18	Santosh Kumar	54			
	148	Karthik Reddy	53			
	169	Selvam Kumar	52			
	105	Suresh Yadav	52			
	129	Deepak Reddy	50			
	135	Prakash Singh	50			

Business Insight

Partners completing the most successful deliveries are key contributors to operational efficiency.

Recommendation

- Recognize high-performing partners.
- Balance workloads across the delivery team.

Q3 Which partners are associated with the most cancellations or delays?

SELECT

d.partner_id,

d.partner_name,

SUM(o.status = 'Cancelled') AS total_cancellations,

ROUND(AVG(TIMESTAMPDIFF(MINUTE,

o.order_time,

o.delivered_time)),2) AS average_delivery_time

FROM orders o

INNER JOIN delivery_partners d

ON o.partner_id = d.partner_id

GROUP BY

d.partner_id,

d.partner_name

ORDER BY

total_cancellations DESC,

average_delivery_time DESC;

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	partner_id	partner_name	total_cancellations	average_delivery_time
▶	31	Sriram Pillai	14	57.35
	118	Deepak Patel	13	52.76
	88	Manoj Kumar	12	56.90
	174	Harish Pillai	12	53.71
	26	Ravi Kumar	12	50.23
	75	Deepak Sharma	11	54.64
	64	Mani Sharma	11	53.29
	180	Krishnan Pillai	11	52.93
	10	Ramesh Sharma	11	51.65
	35	Vinod Gowda	11	50.20

Business Insight

Partners with high cancellation counts or longer delivery times may require operational support or performance improvements.

Recommendation

- Review partner performance regularly.
- Provide additional training.
- Optimize route allocation.

Q4 What is the average customer rating received per delivery partner?

SELECT

d.partner_id,

d.partner_name,

ROUND(AVG(r.customer_rating), 2) AS average_rating

FROM delivery_partners d

INNER JOIN orders o

ON d.partner_id = o.partner_id

INNER JOIN ratings r

ON o.order_id = r.order_id

GROUP BY

d.partner_id,

d.partner_name

ORDER BY average_rating DESC;

Result Grid				Filter Rows:	Export:	Wrap Cell Content:
	partner_id	partner_name	average_rating			
▶	62	Harish Nair	3.55			
	92	Harish Yadav	3.43			
	5	Balu Pillai	3.39			
	8	Arumugam Nair	3.36			
	143	Thiru Kumar	3.34			
	147	Ramesh Patel	3.34			
	61	Harish Nair	3.32			
	195	Manoj Sharma	3.32			
	200	Gopal Pillai	3.31			
	16	Selvam Gowda	3.30			

Business Insight

Higher customer ratings indicate better delivery service quality and customer satisfaction.

Recommendation

- Reward highly rated partners.
- Coach lower-rated partners to improve service quality.

10. Recommendations

The following business recommendations are proposed.

Recommendation 1

Increase delivery partner availability during peak hours.

Expected Result:

Reduced delivery delays.

Recommendation 2

Improve restaurant preparation efficiency.

Expected Result:

Faster deliveries.

Recommendation 3

Launch customer loyalty programs.

Expected Result:

Higher customer retention.

Recommendation 4

Optimize delivery routes.

Expected Result:

Lower operational cost.

Recommendation 5

Expand services in high-revenue cities.

Expected Result:

Increased company revenue.

Recommendation 6

Monitor cancellation trends monthly.

Expected Result:

Reduced cancellation rates.

11. Conclusion

The Smart Delivery Operations Analytics System successfully demonstrates the practical application of SQL in solving real-world business problems. Through relational database design, data import, validation, cleaning, and analytical querying, meaningful insights were generated to improve delivery operations, customer satisfaction, and revenue performance.

The project showcases a broad range of SQL concepts—from basic queries to advanced analytical techniques such as joins, subqueries, Common Table Expressions (CTEs), window functions, views, and stored procedures. The analysis supports informed decision-making and provides actionable recommendations for business improvement.

Overall, this project strengthened practical skills in database management, SQL analytics, and business reporting, while illustrating how data can be transformed into valuable insights for organizational growth.